API REFERENCE

HORIZON GRID

API Documentation

Version: 0.1.0

Documentation prepared: 2026-08-17

PREPARED FOR EVALUATION

Table of Contents

HORIZON GRID — API Reference

Every Signal. One Operational Picture.

About this document

This is a complete, standalone reference to the HTTP API that powers HORIZON GRID: every route the backend actually exposes, traced directly to the FastAPI route files that define it (`backend/app/api/routes/*.py`) and to the Pydantic schemas those routes validate against. It is written for two audiences: an engineer integrating another system (a SIEM, a SOAR playbook, a script) against HORIZON GRID directly, and anyone who wants to understand exactly what the web UI is doing under the hood, because it is doing nothing the UI itself couldn't also do by calling these same endpoints.

Nothing in this document is inferred from a comment or a docstring's stated intent — every method, path, permission requirement, request field, response field, and error code below was confirmed by reading the route function and the schema class it depends on. Where the existing internal API chapter (`backend-02-api-reference.md`) was checked against the current route files for this document, three gaps were found and are filled in here: `POST /api/v1/lookup/{lookup_id}/export` (defined in `lookup.py` but previously undocumented), the entire `routes/admin.py` router (7 endpoints, user management), and the entire `routes/security_assessment.py` router (5 endpoints, the Security Assessment Toolkit's API surface) — both of the latter two were explicitly marked out of scope in that chapter's own introduction. All three are documented in full below, alongside everything already covered.

Why an API, and why two separate services

HORIZON GRID ships as two independent services: a FastAPI backend (Python) and a Next.js frontend (TypeScript/React), running as separate containers that talk to each other exclusively over HTTP. The frontend holds no business logic of its own worth calling "the platform" — it renders pages and calls the backend's API for every piece of real data or work: running an investigation, fetching a case, testing a provider credential, reading the dashboard's KPIs. The backend has no concept of pages, sessions, or HTML; it is a stateless JSON-over-HTTP service that authenticates every single request independently via a bearer token, and does nothing else.

This separation is not incidental — it is the reason this document is useful at all. Because the backend doesn't trust or depend on the frontend in any way, everything the web UI can do, this API can do too, for anyone holding a valid token with the right permission: a SOAR platform can call `POST /api/v1/lookup/stream` directly to trigger an investigation as part of an automated playbook; a script can poll `GET /api/v1/dashboard/kpis` on a schedule and feed it into an existing monitoring stack; an analyst's own tooling can pull `GET /api/v1/cases` without opening a browser at all. The web UI is simply the first, but not the only, client of this API.

[MISSING FIGURE: standalone-api-documentation-diagram-1.png]

Base URL and versioning

Every endpoint in this reference is served by the backend container. On this install, that is:

```
http://localhost:8000
```

Every route documented below is mounted under one fixed prefix, `/api/v1` (`settings.api_v1_prefix` , `backend/app/core/config.py`) — so, for example, the KPI endpoint's full URL is `http://localhost:8000/api/v1/dashboard/kpis` . This host, port, and prefix are what this specific installation is actually configured to use; the port is one of the values the Windows Setup Wizard lets an administrator change at install time (see the Installation Guide), so a different install could genuinely be reachable at a different port. There is no `/api/v2` — every route in the running application today lives under `v1` , and nothing in the codebase currently plans a breaking change that would require one.

Two more, deliberately unauthenticated, self-describing endpoints exist for exploring the API interactively rather than reading this document:

- `GET /docs` — FastAPI's built-in Swagger UI. It renders every route below with a live "Try it out" form, including an "Authorize" button that accepts a bearer token for the rest of the session.
- `GET /api/v1/openapi.json` — the raw OpenAPI 3.1 schema Swagger UI itself is built from, useful for generating a typed client in another language.

Authentication: obtaining and using a JWT

HORIZON GRID authenticates every protected request with a JSON Web Token (JWT), sent as a standard `Authorization: Bearer <token>` header — there are no cookies, no sessions, and no API keys of the platform's own for this purpose (provider/AI-backend API keys are a completely separate thing, covered below in §3–4 and §10, and are used server-side to call third-party services, never to call HORIZON GRID itself).

To obtain a token, exchange an email and password for a token pair at `POST /api/v1/auth/login` (full detail in §1 below):

```
curl -X POST http://localhost:8000/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email": "analyst@example.com", "password": "<REDACTED>"}'
```

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIs...<REDACTED>",
  "refresh_token": "eyJhbGciOiJIUzI1NiIs...<REDACTED>",
  "token_type": "bearer"
}
```

Every subsequent call attaches the access token:

```
curl http://localhost:8000/api/v1/dashboard/kpis \
  -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIs...<REDACTED>"
```

Two distinct token types are issued together, signed HS256 (`backend/app/auth/security.py`), and are not interchangeable — a route that expects an access token rejects a refresh token presented in its place, and vice versa:

Token	`type` claim	Default lifetime	Used for
Access token	`"access"`	30 minutes (`access_token_expire_minutes`)	The `Authorization: Bearer` header on every protected API call
Refresh token	`"refresh"`	7 days (`refresh_token_expire_days`)	Exchanged at `POST /api/v1/auth/refresh` for a brand-new access/refresh pair, without re-sending a password

Both lifetimes are configurable per-install and are not hardcoded absolutes; the values above are the shipped defaults. An access token also embeds the user's `role` and a `token_version` counter — the latter means an administrator resetting a user's password immediately invalidates every token already issued to that user, not just future logins, because `get_current_user` re-checks `token_version` against the live database value on every single request (it also re-checks `is_active` the same way, so disabling an account takes effect on that account's very next request, not at token expiry).

There is no logout endpoint and no server-side token revocation list beyond the `token_version` mechanism above — an access token, once issued, remains valid for up to its own 30-minute lifetime even if the browser tab is closed. This is a documented, known characteristic of the current design, not an oversight to route around.

Authorization: roles and permissions

Every protected route is gated by one specific *permission string*, checked by a single shared dependency, `require_permission("<permission>")` (`backend/app/auth/rbac.py`). There are exactly three roles (`backend/app/models/user.py` 's `Role` enum) and eighteen distinct permission strings checked anywhere in the API:

Role	Can do
ADMIN	Everything below — every permission string in the matrix.
ANALYST	Run and read investigations, export reports, read evidence, generate every AI explanation, run threat-hunting/detection-rule generation, manage their own basket, create/read/write/close cases, run and read Security Assessment Toolkit scans, read the dashboard. **Cannot** manage provider/AI credentials or manage user accounts.
VIEWER	Read investigations, read evidence, read cases, read Security Assessment Toolkit findings, read the dashboard — deliberately broad *read* access to the whole operational picture. **Cannot** create investigations, export anything, run a Security Assessment scan, or perform any create/write/manage/generate action anywhere in the API.

The full permission-to-permission-string matrix, reproduced here because every endpoint below cites one of these strings by name:

Permission string	admin	analyst	viewer
`lookup:create`	✓	✓	
`lookup:read`	✓	✓	✓
`lookup:export`	✓	✓	
`evidence:read`	✓	✓	✓
`analysis:generate`	✓	✓	
`copilot:query`	✓	✓	
`hunting:generate`	✓	✓	
`basket:manage`	✓	✓	
`case:create` / `case:write` / `case:close`	✓	✓	
`case:read`	✓	✓	✓
`security_assessment:create`	✓	✓	
`security_assessment:read`	✓	✓	✓
`dashboard:read`	✓	✓	✓
`provider:manage`	✓		
`audit:read`	✓		
`user:manage`	✓		

A request with no token, an expired token, or a token failing any of the checks in the previous section always fails the same way, regardless of which route it hit:

```
HTTP 401
{"detail": "Could not validate credentials"}
```

A request with a *valid* token whose role's permission set doesn't include the route's required permission fails like this — naming both the caller's own role and the specific missing permission, which is safe to show since the caller already knows its own role from its own token:

```
HTTP 403
{"detail": "Role 'viewer' lacks permission 'lookup:create'"}
```

This document states the one required permission string next to every endpoint below; it does not repeat the two error shapes above on every single endpoint, since they are identical everywhere.

[MISSING FIGURE: standalone-api-documentation-diagram-2.png]

Conventions used in this reference

- **IDs.** Every resource ID (`lookup_id` , `case_id` , `item_id` , `run_id` , `user_id` , …) is a UUID, serialized as a string in every JSON response.
- **Timestamps.** Every persisted timestamp is serialized with `.isoformat()` — an ISO-8601 string, always UTC-aware where the underlying column is timezone-aware.
- **Validation errors.** Any request body that fails Pydantic validation (missing required field, wrong type, string too long/short) gets FastAPI's standard `422 Unprocessable Entity` with a `detail` array describing exactly which field and constraint failed. This is not restated per endpoint below unless a route additionally raises a *hand-written* `422` for a business rule Pydantic can't express on its own (e.g. "at least 2 IOCs to compare") — those are called out explicitly.
- **Pagination.** A handful of list endpoints (`GET /lookup` , `GET /cases`) take a `limit` query parameter that is silently clamped server-side to `[1, 200]` rather than rejected if a caller passes something outside that range — there is no `offset` /cursor parameter on either of them; both return the N most recent rows.
- **"None explicit."** Where an endpoint's Errors line says "none explicit," it means the route raises no hand-written `HTTPException` at all — a request that reaches the handler will always get a `200` / `201` / `204` (barring an unrelated infrastructure failure), aside from the universal `401` / `403` / `422` cases described above.
- **Secrets.** Every example in this document uses `<REDACTED>` or `<YOUR_API_KEY>` in place of any value that would be a real credential in production. No real key, password, or token appears anywhere below.

Rate limiting

Exactly one route in the entire API is rate-limited: `POST /api/v1/lookup/stream` , capped per-user at `lookup_rate_limit_max_calls` (10) calls per `lookup_rate_limit_window_seconds` (60) seconds, enforced by a Redis fixed-window counter (`backend/app/core/cache.py`). Every other endpoint in this reference — including authentication itself — has no rate limit of its own; this is a known, documented gap, not a hidden one, called out in the Security Architecture chapter. Exceeding the lookup limit returns:

```
HTTP 429
{"detail": "Rate limit exceeded: max 10 lookups per 60s. Each lookup fans out to every provider plus the crawler and multiple AI calls, so this bounds cost/load
```

Endpoint Reference

§	Router	Base path	Endpoints
1	`routes/auth.py`	`/api/v1/auth`	4
2	`routes/lookup.py`	`/api/v1/lookup`	6
3	`routes/providers.py`	`/api/v1/providers`	2
4	`routes/ai_config.py`	`/api/v1/ai`	2
5	`routes/analysis.py`	`/api/v1/lookup/{lookup_id}/analysis`	10
6	`routes/hunting.py`	`/api/v1/lookup/{lookup_id}`	2
7	`routes/pivot.py`	`/api/v1/lookup/{lookup_id}/pivots`	1
8	`routes/basket.py`	`/api/v1/basket`	5
9	`routes/cases.py`	`/api/v1/cases`	8
10	`routes/runtime.py`	`/api/v1/runtime`	10
11	`routes/admin.py`	`/api/v1/admin`	7
12	`routes/security_assessment.py`	`/api/v1/security-assessment`	5
13	`routes/dashboard.py`	`/api/v1/dashboard`	2

Total: 64 endpoints under `/api/v1`, plus three unversioned utility routes covered in the final section. Routers are registered in `backend/app/main.py` in exactly this order (auth → lookup → providers → ai_config → analysis → hunting → pivot → basket → cases → runtime → admin → security_assessment → dashboard); FastAPI's routing is path-based, not order-sensitive, so this order affects only where each router appears below, not how requests are matched.

---

1. Authentication — `/api/v1/auth`

All four endpoints are unauthenticated *at the dependency level* except `/me`; `/register` and `/login` are the only two ways to ever obtain a token.

`POST /api/v1/auth/register`

- **Full URL:** `http://localhost:8000/api/v1/auth/register`
- **Permission:** none (public) — but see the bootstrap rule below.
- **Purpose:** Register a new account. **Bootstrap-only:** the very first account ever created on a fresh instance is automatically granted the `admin` role; every registration attempt after that is rejected outright, not silently downgraded to `analyst`. This is the platform's only way to obtain an initial administrator — after that first admin exists, every further account must be created by an existing admin via `POST /api/v1/admin/users` (§11), not through this endpoint.
- **Request body:**

Field	Type	Notes
`email`	`EmailStr`	required
`password`	`str`	8–72 characters (72 is bcrypt's real effective limit)
`full_name`	`str`	optional, defaults to `""`

- **Response (`201`):** `{"id": str, "email": str, "full_name": str, "role": "admin"}`
- **Errors:** `400` `"Email already registered"`; `403` `"Self-registration is closed. Ask an administrator to create your account from the Administration page."` once at least one user already exists.
- **Example:**

```
curl -X POST http://localhost:8000/api/v1/auth/register \
  -H "Content-Type: application/json" \
  -d '{"email": "first.admin@example.com", "password": "<REDACTED>", "full_name": "First Admin"}'
```

`POST /api/v1/auth/login`

- **Full URL:** `http://localhost:8000/api/v1/auth/login`
- **Permission:** none (public).

- **Purpose:** Exchange an email/password for a fresh access/refresh token pair.
- **Request body:** `{"email": "EmailStr", "password": "str (max 72 chars)"}`
- **Response:** `{"access_token": str, "refresh_token": str, "token_type": "bearer"}`
- **Errors:** `401` `"Invalid email or password"`; `403` `"Account disabled"` if the account's `is_active` is false.
- **Example:** see "Authentication" above.

`POST /api/v1/auth/refresh`

- **Full URL:** `http://localhost:8000/api/v1/auth/refresh`
- **Permission:** none at the dependency level — the refresh token itself is the credential, decoded and validated inline.
- **Purpose:** Exchange a still-valid refresh token for a brand-new access/refresh pair, without a password.
- **Request body:** `{"refresh_token": "str"}`
- **Response:** identical shape to `/login`.
- **Errors:** `401` `"Invalid refresh token"` — covers a missing/malformed token, a token whose `type` claim isn't `"refresh"`, an unknown or inactive user, and a token whose embedded `token_version` no longer matches the database (e.g. the password was reset since this token was issued).
- **Example:**

```
curl -X POST http://localhost:8000/api/v1/auth/refresh \
  -H "Content-Type: application/json" \
  -d '{"refresh_token": "eyJhbGciOiJIUzI1NiIs...<REDACTED>"}'
```

`GET /api/v1/auth/me`

- **Full URL:** `http://localhost:8000/api/v1/auth/me`
- **Permission:** valid access token required (no specific permission string beyond `get_current_user`).
- **Purpose:** Return the identity associated with the presented token — useful for a client to confirm who it's authenticated as and what role it holds, without decoding the JWT itself.
- **Response:** `{"id": str, "email": str, "full_name": str, "role": "admin"|"analyst"|"viewer"}`. Note: `full_name` is hard-coded to `""` on this specific route rather than read from the database — a real, minor inconsistency with `/admin/users`, which does return the real value.
- **Errors:** `401` `"Could not validate credentials"`.
- **Example:**

```
curl http://localhost:8000/api/v1/auth/me \
  -H "Authorization: Bearer <REDACTED>"
```

---

2. IOC Lookup — `/api/v1/lookup`

The core investigation pipeline: create a lookup, stream its lifecycle, re-run the AI verdict against a different backend, export a report, and read results back. This is the single endpoint group HORIZON GRID's search box drives end to end.

`POST /api/v1/lookup/stream`

- **Full URL:** `http://localhost:8000/api/v1/lookup/stream`
- **Permission:** `lookup:create` (admin, analyst)
- **Purpose:** Create a lookup, classify the submitted string into a typed IOC, fan it out concurrently to every applicable provider, and stream the entire investigation back to the caller as Server-Sent Events (SSE) as each step completes — this is the one HTTP call behind an entire investigation.
- **Request body:**

Field	Type	Notes
`value`	`str`	the raw IOC string, e.g. `"185.220.101.45"`
`ioc_type_hint`	`str | null`	forces type classification when auto-detection would be ambiguous
`provider_ids`	`list[str] | null`	restricts the fan-out to specific providers; omit for every applicable provider
`ai_backend`	`str | null`	overrides the platform's active AI backend for this one investigation only

- **Response:** `Content-Type: text/event-stream` — not a single JSON body. Named SSE events, always in this order:

Event	Payload
`detected`	`{lookup_id, ioc_value, ioc_type}`
`provider_result` (×N)	one per provider, as it finishes — `{provider_id, provider_name, category, status, data, source_url, error_message, latency_ms}`
`provider_summary` (×N, only for providers that returned real data)	that provider's AI-generated one-paragraph summary
`correlation`	`{nodes: [...], edges: [...]}` — the relationship graph built from every provider result
`final_assessment`	the full AI verdict object — `executive_summary`, `technical_summary`, `threat_assessment`, `supporting_evidence`, `agreeing_providers`, `disagreeing_providers`, `relationships_summary`, `mitre_mappings`, `risk: {overall_risk_score, confidence_score, malicious_probability, severity}`, `final_verdict`, `ai_backend`, `ai_model`, `ai_outcome`, and more
`done`	`{lookup_id}`

On a mid-stream failure, an `error` event (`{"message": str}`) is emitted in place of `final_assessment` / `done` , and the lookup's status is persisted as `failed` — a client must inspect the event stream itself to detect failure; the initial HTTP response code is always `200` once streaming begins.

- **Errors:** `429` (see Rate Limiting above); `422` `"Could not determine IOC type; pass ioc_type_hint."` if automatic type detection fails and no hint was supplied.
- **Example:**

```
curl -N -X POST http://localhost:8000/api/v1/lookup/stream \
  -H "Authorization: Bearer <REDACTED>" \
  -H "Content-Type: application/json" \
  -d '{"value": "185.220.101.45"}'
```

```
event: detected
data: {"lookup_id": "b3c1...", "ioc_value": "185.220.101.45", "ioc_type": "ipv4"}

event: provider_result
data: {"provider_id": "abuseipdb", "provider_name": "AbuseIPDB", "category": "threat_intel", "status": "ok", "data": {...}, "latency_ms": 412}

event: final_assessment
data: {"final_verdict": "malicious", "risk": {"overall_risk_score": 82.4, "confidence_score": 61.0, "malicious_probability": 82.4, "severity": "critical"}, ...}

event: done
data: {"lookup_id": "b3c1..."}
```

`POST /api/v1/lookup/{lookup_id}/reanalyze`

- **Full URL:** `http://localhost:8000/api/v1/lookup/{lookup_id}/reanalyze`
- **Permission:** `lookup:create` (admin, analyst)
- **Purpose:** Re-run only the final-assessment AI step against an already-completed lookup's persisted evidence, using an explicitly chosen AI backend — the "compare Claude vs. Groq on the same evidence" feature. No provider is re-queried. The deterministic score (see the Threat Scoring reference) is *recomputed* from the same persisted evidence every backend comparison shares, so every backend shown side-by-side displays the identical score, differing only in AI-authored prose. The result is stored as an additional, non-primary assessment — the lookup's original verdict is untouched.
- **Path param:** `lookup_id` (UUID)
- **Request body:** `{"ai_backend": "str"}` — e.g. `"anthropic"`, `"groq"`, `"ollama"`, `"gemini"`, `"bedrock"`.
- **Response:** `{"id": str, "ai_backend": str, "ai_model": str | null, "assessment": {...same shape as final_assessment above...}}`
- **Errors:** `404` `"Lookup not found"`; `400` `"Lookup must be completed before it can be re-analyzed."`
- **Example:**

```
curl -X POST http://localhost:8000/api/v1/lookup/b3c1.../reanalyze \
  -H "Authorization: Bearer <REDACTED>" \
  -H "Content-Type: application/json" \
  -d '{"ai_backend": "groq"}'
```

`GET /api/v1/lookup/{lookup_id}/assessments`

- **Full URL:** `http://localhost:8000/api/v1/lookup/{lookup_id}/assessments`
- **Permission:** `lookup:read` (admin, analyst, viewer)
- **Purpose:** List every final assessment ever generated for a lookup — the original plus every subsequent re-analysis — so a client can render a side-by-side "backend A said X, backend B said Y" comparison.
- **Response:** list of `{id, ai_backend, ai_model, is_primary: bool, assessment: {...}, created_at}`, ordered oldest first.
- **Errors:** none explicit — an unknown `lookup_id` returns an empty list rather than a `404`.

`GET /api/v1/lookup/{lookup_id}`

- **Full URL:** `http://localhost:8000/api/v1/lookup/{lookup_id}`
- **Permission:** `lookup:read` (admin, analyst, viewer)
- **Purpose:** Fetch full detail for a single investigation — every provider result, every AI summary, the primary final assessment, and the correlation graph, rebuilt from Postgres alone (so this works correctly even long after the live SSE stream that produced it has ended).
- **Response:** `{id, ioc_value, ioc_type, status, final_verdict, risk_score, confidence_score, final_assessment, provider_results: [...], ai_summaries: [...], created_at, correlation: {nodes: [{node_id, ioc_type, value, labels: []}], edges: [{source, target, relationship, confidence, provenance}]}}`. The correlation payload always includes the seed IOC as a node, even with zero edges, so "no relationships found" is distinguishable from "graph data unavailable."
- **Errors:** `404` `"Lookup not found"`.
- **Example:**

```
curl http://localhost:8000/api/v1/lookup/b3c1... \
  -H "Authorization: Bearer <REDACTED>"
```

`POST /api/v1/lookup/{lookup_id}/export`

Not present in the prior internal API chapter — added here after confirming it directly in `backend/app/api/routes/lookup.py`.

- **Full URL:** `http://localhost:8000/api/v1/lookup/{lookup_id}/export?format=pdf`
- **Permission:** `lookup:export` (admin, analyst — deliberately **not** viewer; exporting a file to disk is treated as a materially more sensitive action than viewing the same data in the browser).
- **Purpose:** Server-render a completed investigation as a downloadable PDF or CSV report. (The other two export formats offered by the UI's export menu, Markdown and JSON, are built entirely client-side from data the browser already has and never call this or any other endpoint.)
- **Query param:** `format` — `"pdf"` or `"csv"`, required.
- **Response:** the raw file bytes, with `Content-Type: application/pdf` or `text/csv` and a `Content-Disposition: attachment; filename="ioc-assessment-<lookup_id>.<format>"` header.
- **Security note, because it's genuinely relevant to anyone building on this endpoint:** both formats carry real, tested protections against injection via attacker-influenced free text (an IOC value or AI-generated field that starts with `=` / `+` / `-` / `@`, which spreadsheet software would otherwise execute as a live formula on open; or one containing `<` / `>` / `&`, which a naive PDF renderer would otherwise parse as markup rather than display as text). Both were real, fixed issues this project tracked as BUG-022/023, not theoretical concerns.
- **Errors:** `404` `"Lookup not found"`.
- **Example:**

```
curl -X POST "http://localhost:8000/api/v1/lookup/b3c1.../export?format=pdf" \
  -H "Authorization: Bearer <REDACTED>" \
  -o ioc-assessment-b3c1....pdf
```

`GET /api/v1/lookup`

- **Full URL:** `http://localhost:8000/api/v1/lookup?limit=50`
- **Permission:** `lookup:read` (admin, analyst, viewer)
- **Purpose:** List recent investigations across the whole team — this is a shared, not per-analyst, view; every role with `lookup:read` sees every investigation anyone has run.
- **Query param:** `limit` — default `50`, clamped server-side to `[1, 200]`.
- **Response:** list of `{id, ioc_value, ioc_type, status, final_verdict, risk_score, confidence_score, created_at}`, newest first.
- **Errors:** none explicit.

---

3. Providers — `/api/v1/providers`

Read-only health reporting for the 18 registered IOC providers, plus a live credential test used by the setup wizard and the Manage Providers screen.

`GET /api/v1/providers/health`

- **Full URL:** `http://localhost:8000/api/v1/providers/health`
- **Permission:** `dashboard:read` (admin, analyst, viewer)
- **Purpose:** Report the real, database-backed health of every registered provider across four rolling windows — 1h/24h/7d/30d — computed from actual persisted call outcomes, never a static config check.
- **Response:** a list of per-provider objects (one for every registered provider, including any with zero calls ever): `{provider_id, provider_name, category, configured, requires_key, supported_types: [...], "1h": {...}, "24h": {...}, "7d": {...}, "30d": {...}}`, where each window object is `{status, success_rate, avg_latency_ms, consecutive_failures, rate_limited_count}`.

- **Behavioral rule worth stating explicitly, because it governs how to interpret the output correctly:** `status` is one of `healthy` / `degraded` / `down` / `unknown`. A window with zero real attempts is always `unknown`, never `healthy` — and a provider correctly reporting "nothing found" or "this IOC type doesn't apply to me" counts as a *healthy* outcome, not a failure. `success_rate` / `avg_latency_ms` are `null`, never `0`, when a window has no qualifying attempts. This exact rule was a real, fixed bug during this project.
- **Errors:** none explicit.
- **Example:**

```
curl http://localhost:8000/api/v1/providers/health \
  -H "Authorization: Bearer <REDACTED>"
```

```
[
  {
    "provider_id": "virustotal", "provider_name": "VirusTotal", "category": "threat_intel",
    "configured": true, "requires_key": true,
    "supported_types": ["domain", "ipv4", "ipv6", "md5", "sha1", "sha256", "sha512", "url"],
    "24h": {"status": "healthy", "success_rate": 0.98, "avg_latency_ms": 340, "consecutive_failures": 0, "rate_limited_count": 0}
  }
]
```

`POST /api/v1/providers/{provider_id}/test`

- **Full URL:** `http://localhost:8000/api/v1/providers/{provider_id}/test`
- **Permission:** `provider:manage` (admin only)
- **Purpose:** Live credential check for one IOC provider — makes one real outbound call with the candidate credential supplied in the body. The credential is never persisted by this route; it exists purely to validate a key *before* saving it via `POST /api/v1/runtime/ioc-providers/{provider_id}` (§10).
- **Path param:** `provider_id` — e.g. `"abuseipdb"`.
- **Request body:** `{"credentials": {"api_key": "<YOUR_API_KEY>"}}`
- **Response:** `{"provider_id": str, "ok": bool, "message": str, "latency_ms": number}`
- **Errors:** none raised as an HTTP error — a bad key still returns `200` with `"ok": false` and an explanatory `message`.
- **Example:**

```
curl -X POST http://localhost:8000/api/v1/providers/abuseipdb/test \
  -H "Authorization: Bearer <REDACTED>" \
  -H "Content-Type: application/json" \
  -d '{"credentials": {"api_key": "<YOUR_API_KEY>"}}'
```

---

4. AI Configuration — `/api/v1/ai`

The AI-backend analog of §3: live credential testing and model-list discovery, used by the setup wizard and Manage Providers screen for the five supported AI backends (Ollama, Anthropic, AWS Bedrock, Google Gemini, Groq).

`POST /api/v1/ai/test`

- **Full URL:** `http://localhost:8000/api/v1/ai/test`
- **Permission:** `provider:manage` (admin only)
- **Purpose:** Live credential check for any AI backend — sends one real, minimal chat request with the candidate credentials. Never persists anything.
- **Request body:** `{"backend": "str", "credentials": {"api_key": "<YOUR_API_KEY>"}, "model": "str | null"}`
- **Response:** `{"backend": str, "ok": bool, "message": str, "model": str | null, "latency_ms": number}`
- **Errors:** none raised as an HTTP error — failures surface through `ok` / `message` in a `200` response.
- **Example:**

```
curl -X POST http://localhost:8000/api/v1/ai/test \
  -H "Authorization: Bearer <REDACTED>" \
  -H "Content-Type: application/json" \
  -d '{"backend": "anthropic", "credentials": {"api_key": "<REDACTED>"}, "model": "claude-sonnet-4-5-20250929"}'
```

`POST /api/v1/ai/{backend}/models`

- **Full URL:** `http://localhost:8000/api/v1/ai/{backend}/models`
- **Permission:** `provider:manage` (admin only)
- **Purpose:** Return the model list for the AI-backend picker's dropdown. Groq and Ollama get real, live discovery; Anthropic/Gemini/Bedrock return a curated static list (there's no equivalently simple live-discovery call to make against those APIs from this codebase today).

- **Path param:** `backend` — `"groq"`, `"ollama"`, `"anthropic"`, `"gemini"`, `"bedrock"`, or any other string (returns an empty list).
- **Request body:** `{"credentials": {"api_key": "<YOUR_API_KEY>"}}` (default `{}`)
- **Response:** always includes `backend` and `models`; also `source` (`"live"`, `"static"`, `"fallback"`, or `"unknown"`) and `default`.
- **Errors:** none raised as an HTTP error — a provider-side failure degrades to the fallback/static list rather than failing the request.
- **Example:**

```
curl -X POST http://localhost:8000/api/v1/ai/groq/models \
  -H "Authorization: Bearer <REDACTED>" \
  -H "Content-Type: application/json" \
  -d '{"credentials": {"api_key": "<REDACTED>"}}'
```

```
{"backend": "groq", "models": ["llama-3.3-70b-versatile", "mixtral-8x7b-32768"], "source": "live", "default": "llama-3.3-70b-versatile"}
```

---

5. Lookup Analysis — `/api/v1/lookup/{lookup_id}/analysis`

The "explain the verdict" endpoints: on-demand AI generations grounded in a *completed* lookup's persisted evidence ledger — never the AI's own free association. Every endpoint below except `/evidence` requires the lookup's status to be `completed`; both shared error conditions are stated once here rather than repeated ten times:

- `404` `"Lookup not found"` if the `lookup_id` doesn't exist.
- `409` `"Lookup is not completed yet (status=<status>)"` if it exists but hasn't finished running (every endpoint below except `/evidence`).

`GET /api/v1/lookup/{lookup_id}/analysis/evidence`

- **Permission:** `evidence:read` (admin, analyst, viewer)
- **Purpose:** Return the raw evidence ledger — the "Show Receipts" view. Purely deterministic; no AI call. This is the one endpoint in this section with **no** `409` completed-status gate, since raw evidence can exist and be worth inspecting even mid-investigation.
- **Response:** list of `{id, evidence_type, source_label, provider_id, claim, interpretation, confidence, related_ioc_type, related_ioc_value, source_url, observed_at, created_at}`.
- **Errors:** `404` only.
- **Example:** `curl http://localhost:8000/api/v1/lookup/b3c1.../analysis/evidence -H "Authorization: Bearer <REDACTED>"`

`POST /api/v1/lookup/{lookup_id}/analysis/why`

- **Permission:** `analysis:generate` (admin, analyst)
- **Purpose:** AI explanation of why the IOC is, or isn't, considered malicious.
- **Response:** `{"verdict_restated": str, "reasons": [{"reason": str, "evidence_ids": [str]}], "caveat": str | null}`
- **Example:** `curl -X POST http://localhost:8000/api/v1/lookup/b3c1.../analysis/why -H "Authorization: Bearer <REDACTED>"`

`POST /api/v1/lookup/{lookup_id}/analysis/what-is-this`

- **Permission:** `analysis:generate` (admin, analyst)
- **Purpose:** AI plain-language and technical explanation of what the indicator actually is.
- **Response:** `{"plain_language_summary": str, "technical_explanation": str, "evidence_ids": [str], "confidence_narrative": str, "related_infrastructure": [str]}`

`POST /api/v1/lookup/{lookup_id}/analysis/disagreement`

- **Permission:** `analysis:generate` (admin, analyst)
- **Purpose:** AI summary of where the queried providers agree and disagree.
- **Response:** `{"agreement": str, "conflict": str, "missing_data": str, "most_reliable_evidence": str, "evidence_ids": [str]}`

`POST /api/v1/lookup/{lookup_id}/analysis/false-positive`

- **Permission:** `analysis:generate` (admin, analyst)
- **Purpose:** AI assessment of false-positive likelihood — e.g. is this really a CDN edge or scanner rather than genuinely malicious infrastructure.
- **Response:** `{"likely_false_positive": bool, "candidate_categories": ["cdn"|"cloud_provider"|"shared_hosting"|"nat"|"vpn"|"proxy"|"security_scanner"|"search_crawler"|"monitoring_system"|"legitimate_business_infrastructure"|"none", ...], "explanation": str, "evidence_ids": [str]}`

`POST /api/v1/lookup/{lookup_id}/analysis/challenge`

- **Permission:** `analysis:generate` (admin, analyst)

- **Purpose:** AI "red-team" self-challenge — actively argues against the verdict the platform already produced, rather than just restating it.
- **Response:** `{"supporting_evidence": [{"reason": str, "evidence_ids": [str]}], "contradictory_evidence": [...], "missing_evidence": [str], "alternative_explanation": str, "final_confidence": "low"|"medium"|"high", "final_confidence_rationale": str}`

`POST /api/v1/lookup/{lookup_id}/analysis/next-actions`

- **Permission:** `analysis:generate` (admin, analyst)
- **Purpose:** AI-suggested next investigative steps, informed by the evidence ledger and correlation graph.
- **Response:** `{"actions": [{"action": str, "target_ioc_value": str | null, "target_ioc_type": str | null, "rationale": str, "priority": "low"|"medium"|"high"}]}`

`POST /api/v1/lookup/{lookup_id}/analysis/gaps`

- **Permission:** `analysis:generate` (admin, analyst)
- **Purpose:** AI-identified intelligence gaps — what no provider addressed.
- **Response:** `{"gaps": [{"gap": str, "how_to_close": str}]}`

`POST /api/v1/lookup/{lookup_id}/analysis/score-explanation`

- **Permission:** `analysis:generate` (admin, analyst)
- **Purpose:** AI explanation, in prose, of how the deterministic risk score was derived — the AI is handed the already-final number as a fact and asked only to narrate it; see the Threat Scoring reference for the real mechanism behind the number itself.
- **Response:** `{"components": [{"component": str, "contribution": str, "evidence_ids": [str]}], "summary": str}`

`POST /api/v1/lookup/{lookup_id}/analysis/copilot`

- **Permission:** `copilot:query` (admin, analyst) — the one endpoint in this section gated by a different permission string than `analysis:generate`.
- **Purpose:** Free-form Q&A over a lookup's evidence, correlation graph, and optional analyst-supplied notes.
- **Request body:** `{"question": "str", "notes": ["str", ...]}` — `notes` optional.
- **Response:** `{"answer": str, "evidence_ids": [str], "suggested_follow_ups": [str] (max 4)}`
- **Errors:** `422` `"question is required"` if `question` is empty/whitespace, plus the shared `404` / `409`.
- **Example:**

```
curl -X POST http://localhost:8000/api/v1/lookup/b3c1.../analysis/copilot \
  -H "Authorization: Bearer <REDACTED>" \
  -H "Content-Type: application/json" \
  -d '{"question": "Has this IP been associated with any known ransomware campaign?"}'
```

---

6. Threat Hunting — `/api/v1/lookup/{lookup_id}`

Both endpoints share a lookup-existence check with **no completed-status gate**, unlike §5 — a hunting package or detection rule can be requested even for a lookup still running or one that failed, since the seed IOC value itself is already known the moment the lookup row exists.

`POST /api/v1/lookup/{lookup_id}/hunt`

- **Permission:** `hunting:generate` (admin, analyst)
- **Purpose:** Generate an exact-match plus expansion threat-hunting query package for the seed IOC and its correlated related indicators, across as many detection-platform query languages as requested.
- **Query param:** `formats` — a repeated query param, default `["sigma","splunk_spl","sentinel_kql","elastic","qradar_aql","chronicle_yara_l","suricata","snort","zeek"]`.
- **Response:** `{"exact_match_queries": [{"format": str, "query": str, "detects": str}], "expansion_targets": [{"related_ioc_value": str, "related_ioc_type": str, "rationale": str}], "broader_queries": [{"format": str, "query": str, "detects": str}]}`
- **Errors:** `404` `"Lookup not found"`.
- **Example:** `curl -X POST "http://localhost:8000/api/v1/lookup/b3c1.../hunt?formats=sigma&formats=splunk_spl" -H "Authorization: Bearer <REDACTED>"`

`POST /api/v1/lookup/{lookup_id}/detection`

- **Permission:** `hunting:generate` (admin, analyst)
- **Purpose:** Generate a single detection-rule draft, in one specified format, for the seed IOC and its current verdict/risk score.
- **Query param:** `format` — required; one of `sigma`, `yara`, `splunk_spl`, `sentinel_kql`, `elastic`, `qradar_aql`, `chronicle_yara_l`, `suricata`, `snort`, `zeek`.
- **Response:** `{"format": str, "title": str, "rule": str, "detection_objective": str, "data_source": str, "logic_explanation": str, "false_positive_considerations": str, "severity": "none"|"low"|"medium"|"high"|"critical", "mitre_technique_ids": [str]}`

- **Errors:** `404` `"Lookup not found"`.
- **Example:** `curl -X POST "http://localhost:8000/api/v1/lookup/b3c1.../detection?format=sigma" -H "Authorization: Bearer <REDACTED>"`

---

7. Pivoting — `/api/v1/lookup/{lookup_id}/pivots`

A single, deterministic (non-AI) endpoint — a pure sort over real correlation edges, which by construction can never hallucinate a pivot target.

`GET /api/v1/lookup/{lookup_id}/pivots`

- **Full URL:** `http://localhost:8000/api/v1/lookup/{lookup_id}/pivots?limit=10`
- **Permission:** `lookup:read` (admin, analyst, viewer)
- **Purpose:** Rank every IOC directly related to the seed indicator by confidence and corroboration, so an analyst can jump straight to the most valuable next investigation with one click.
- **Query param:** `limit` — default `10`, clamped server-side to `[1, 50]`.
- **Response:** a ranked list of `{"ioc_value": str, "ioc_type": str, "relationship": str, "confidence": number (0-100), "corroborating_providers": int, "provenance": str, "relevance": "high"|"medium"|"low"}`, sorted by corroborating-provider count first, then confidence.
- **Errors:** `404` `"Lookup not found"`.
- **Example:**

```
curl "http://localhost:8000/api/v1/lookup/b3c1.../pivots?limit=5" \
  -H "Authorization: Bearer <REDACTED>"
```

---

8. IOC Basket — `/api/v1/basket`

A per-analyst scratch space of saved IOCs — every endpoint here operates only on the calling user's own basket; there is no team-shared basket.

`GET /api/v1/basket`

- **Permission:** `basket:manage` (admin, analyst)
- **Purpose:** List the caller's own basket items, newest first.
- **Response:** list of `{id, ioc_value, ioc_type, note, latest_lookup_id, created_at}`.

`POST /api/v1/basket`

- **Permission:** `basket:manage` (admin, analyst)
- **Purpose:** Add an IOC to the caller's basket. Deduplicates by exact `ioc_value` per owner, and best-effort attaches the most recent *completed* lookup for that value, if one exists.
- **Request body:** `{"ioc_value": "str", "ioc_type_hint": "str | null", "note": "str | null"}`
- **Response (`201` for a new item, or `200`-shaped body for one that already existed):** `{id, ioc_value, ioc_type, note, latest_lookup_id, created_at}`
- **Errors:** `422` `"Could not determine IOC type; pass ioc_type_hint."`
- **Example:**

```
curl -X POST http://localhost:8000/api/v1/basket \
  -H "Authorization: Bearer <REDACTED>" \
  -H "Content-Type: application/json" \
  -d '{"ioc_value": "evil-domain.example", "note": "Seen in phishing campaign, ticket #4821"}'
```

`DELETE /api/v1/basket/{item_id}`

- **Permission:** `basket:manage` (admin, analyst)
- **Purpose:** Remove one basket item owned by the caller.
- **Response:** `204 No Content`.
- **Errors:** `404` `"Basket item not found"` — also returned if the item exists but belongs to a different user (ownership mismatch is indistinguishable from non-existence here).

`DELETE /api/v1/basket`

- **Permission:** `basket:manage` (admin, analyst)
- **Purpose:** Clear every one of the caller's own basket items in one call.
- **Response:** `204 No Content`.
- **Errors:** none explicit.

`POST /api/v1/basket/compare`

- **Permission:** `basket:manage` (admin, analyst)
- **Purpose:** Build a deterministic side-by-side comparison table across 2–10 already-investigated IOCs, then generate an AI narrative comparing them.
- **Request body:** `{"lookup_ids": ["str", ...]}`
- **Response:** `{"rows": [{"ioc_value": str, "ioc_type": str, "verdict": str, "risk_score": number, "confidence_score": number, "asn": [str], "malware_families": [str], "threat_actors": [str], "related_domains": [str], "first_seen": str}], "narrative": {"most_dangerous_ioc_value": str | null, "narrative": str, "key_differences": [str]}}`
- **Errors:** `422` `"Provide at least 2 lookup_ids to compare"`; `422` `"Cannot compare more than 10 IOCs at once"`; `422` `"Fewer than 2 of the given lookup_ids resolved to completed lookups"`.
- **Example:**

```
curl -X POST http://localhost:8000/api/v1/basket/compare \
  -H "Authorization: Bearer <REDACTED>" \
  -H "Content-Type: application/json" \
  -d '{"lookup_ids": ["b3c1...", "f772..."]}'
```

---

9. Case Management — `/api/v1/cases`

Cases are team-shared (not per-analyst) investigation containers grouping IOCs, notes, and reports. Most endpoints share one loader that raises `404` `"Case not found"` for a missing case — stated once here.

`GET /api/v1/cases`

- **Permission:** `case:read` (admin, analyst, viewer)
- **Query params:** `status_filter` (optional — `open` / `investigating` / `contained` / `resolved` / `false_positive` / `closed`), `limit` (default `50`, clamped to `[1, 200]`).
- **Response:** list of `{id, title, severity, status, tags, created_at}`.

`POST /api/v1/cases`

- **Permission:** `case:create` (admin, analyst)
- **Purpose:** Create a new case, owned by the calling analyst.
- **Request body:**

Field	Type	Notes
`title`	`str`	1–255 characters
`description`	`str | null`	—
`severity`	`str`	default `"medium"`; one of `low` / `medium` / `high` / `critical`
`tags`	`list[str]`	default `[]`

- **Response (`201`):** the full case object — `{id, title, description, analyst_id, severity, status, tags, created_at, updated_at, iocs: [], notes: [], reports: []}`
- **Example:**

```
curl -X POST http://localhost:8000/api/v1/cases \
  -H "Authorization: Bearer <REDACTED>" \
  -H "Content-Type: application/json" \
  -d '{"title": "Suspected C2 infrastructure — Q3 phishing wave", "severity": "high", "tags": ["phishing", "c2"]}'
```

`GET /api/v1/cases/{case_id}`

- **Permission:** `case:read` (admin, analyst, viewer)
- **Response:** the full case object, as above.
- **Errors:** `404` `"Case not found"`.

`PATCH /api/v1/cases/{case_id}`

- **Permission:** `case:write` (admin, analyst)
- **Purpose:** Partially update a case — only fields explicitly present in the request body are applied.
- **Request body (all optional):** `{"title": str, "description": str, "severity": str, "status": str, "tags": [str]}`

- **Response:** the full updated case object.
- **Errors:** `404` `"Case not found"`.

`POST /api/v1/cases/{case_id}/close`

- **Permission:** `case:close` (admin, analyst) — a distinct permission string from `case:write`, so a role matrix could restrict closing more tightly than editing (today, both are granted to the same two roles).
- **Purpose:** Force-close a case regardless of its current status.
- **Response:** the full updated case object.
- **Errors:** `404` `"Case not found"`.

`POST /api/v1/cases/{case_id}/iocs`

- **Permission:** `case:write` (admin, analyst)
- **Purpose:** Attach an IOC to a case, optionally linking it to an existing lookup.
- **Request body:** `{"ioc_value": "str", "ioc_type": "str", "lookup_id": "str | null"}`
- **Response (`201`):** the full updated case object.
- **Errors:** `404` `"Case not found"`. **Known gap:** supplying a `lookup_id` that isn't a syntactically valid UUID raises an unhandled `ValueError` (an uncaught server error, not a clean `422`) — a real, currently-existing rough edge worth knowing about if you're calling this endpoint programmatically; always send either a real UUID string or `null`.

`DELETE /api/v1/cases/{case_id}/iocs/{ioc_id}`

- **Permission:** `case:write` (admin, analyst)
- **Response:** `204 No Content`.
- **Errors:** `404` `"Case IOC not found"`.

`POST /api/v1/cases/{case_id}/notes`

- **Permission:** `case:write` (admin, analyst)
- **Purpose:** Add an analyst note to a case, optionally anchored to a specific artifact (a free string pair, not a foreign key — e.g. anchoring a note to a particular IOC or graph node).
- **Request body:** `{"body": "str (min 1 char)", "anchor_type": "str | null", "anchor_ref": "str | null"}`
- **Response (`201`):** the full updated case object.
- **Errors:** `404` `"Case not found"`.

---

10. Runtime Configuration — `/api/v1/runtime`

The API surface behind the "Manage Providers" screen: configuring and activating AI backends, configuring and enabling IOC providers, and reading the resulting audit trail — all without editing a `.env` file or restarting a container. Distinct from §3/§4 (which only *test* candidate credentials): this router is what actually *persists* a validated credential.

`GET /api/v1/runtime/ai-providers`

- **Permission:** `provider:manage` (admin only)
- **Purpose:** List every configured AI-backend row — persisted configuration, masked credentials, active/last-test status.
- **Response:** a list of provider-config rows (masked credential fields, never raw secrets).

`POST /api/v1/runtime/ai-providers/{backend}`

- **Permission:** `provider:manage` (admin only)
- **Purpose:** Persist a validated credential and/or model choice for one AI backend. Takes effect on the very next AI call platform-wide — no restart.
- **Path param:** `backend` — one of the five supported backend identifiers.
- **Request body:** `{"credentials": {"api_key": "<YOUR_API_KEY>"}, "model_id": "str | null"}`
- **Errors:** `400` `"Unknown AI backend '<backend>'"`.
- **Example:**

```
curl -X POST http://localhost:8000/api/v1/runtime/ai-providers/anthropic \
  -H "Authorization: Bearer <REDACTED>" \
  -H "Content-Type: application/json" \
  -d '{"credentials": {"api_key": "<REDACTED>"}, "model_id": "claude-sonnet-4-5-20250929"}'
```

`POST /api/v1/runtime/ai-active`

- **Permission:** `provider:manage` (admin only)
- **Purpose:** Switch the platform-wide active AI backend immediately — affects the very next investigation and every subsequent AI call, with no restart.
- **Request body:** `{"backend": "str"}`
- **Response:** `{"active_backend": str}`
- **Errors:** `400` with the underlying validation message if the backend is unknown or hasn't been configured yet.

`GET /api/v1/runtime/ai-active`

- **Permission:** `lookup:read` (admin, analyst, viewer) — deliberately looser than every other endpoint in this router, since read-only callers (e.g. the home page's AI quick-switch widget) don't need `provider:manage` just to *display* the current backend.
- **Response:** `{"backend": str | null, "model_id": str | null}` — both `null` if nothing has ever been configured.

`POST /api/v1/runtime/ai-providers/{backend}/record-test`

- **Permission:** `provider:manage` (admin only)
- **Purpose:** Record the outcome of a prior `POST /api/v1/ai/test` call against a now-saved credential, so the UI can show "last tested: ok, 3 minutes ago" without re-testing on every page load.
- **Request body:** `{"ok": bool, "message": "str"}`
- **Response:** `{"recorded": true}`

`GET /api/v1/runtime/ioc-providers`

- **Permission:** `provider:manage` (admin only)
- **Purpose:** List every one of the 18 known IOC providers merged with its runtime configuration (or a synthesized default if never configured), plus static metadata (`requires_key` , `credential_fields` , `category` , `supported_types`).
- **Response:** a list of provider-config rows, each including `requires_key` , `credential_fields` , `category` , `supported_types` regardless of whether a persisted row exists yet.

`POST /api/v1/runtime/ioc-providers/{provider_id}`

- **Permission:** `provider:manage` (admin only)
- **Purpose:** Persist a validated credential and/or extra configuration for one IOC provider.
- **Path param:** `provider_id` — e.g. `"virustotal"` .
- **Request body:** `{"credentials": {"api_key": "<YOUR_API_KEY>"}, "extra_config": {} | null}`
- **Errors:** `404` `"Unknown IOC provider '<provider_id>'"` .
- **Example:**

```
curl -X POST http://localhost:8000/api/v1/runtime/ioc-providers/virustotal \
  -H "Authorization: Bearer <REDACTED>" \
  -H "Content-Type: application/json" \
  -d '{"credentials": {"api_key": "<REDACTED>"}}'
```

`POST /api/v1/runtime/ioc-providers/{provider_id}/enabled`

- **Permission:** `provider:manage` (admin only)
- **Purpose:** Enable or disable an IOC provider at runtime — takes effect on the next investigation started after this call; historical data for the provider is never touched.
- **Request body:** `{"enabled": bool}`
- **Response:** `{"provider_id": str, "enabled": bool}`

`POST /api/v1/runtime/ioc-providers/{provider_id}/record-test`

- **Permission:** `provider:manage` (admin only)
- **Request body:** `{"ok": bool, "message": "str"}`
- **Response:** `{"recorded": true}`

`GET /api/v1/runtime/audit-log`

- **Permission:** `audit:read` (admin only) — the only route gated by this specific permission string.
- **Purpose:** Retrieve the runtime-configuration audit log — who changed which provider or AI setting, and when.
- **Query param:** `limit` — default `200` .
- **Response:** a list of audit-entry rows.
- **Example:** `curl "http://localhost:8000/api/v1/runtime/audit-log?limit=50" -H "Authorization: Bearer <REDACTED>"`

---

11. Administration — `/api/v1/admin`

Not present in the prior internal API chapter, which explicitly scoped this router out. Added here in full after reading `backend/app/api/routes/admin.py` directly. The backend for the RBAC admin console — every route gated by `user:manage`, granted to `admin` only, and every route re-derives the caller's role from the database on every request rather than trusting the JWT's embedded role claim alone.

`GET /api/v1/admin/users`

- **Full URL:** `http://localhost:8000/api/v1/admin/users`
- **Permission:** `user:manage` (admin only)
- **Purpose:** Search and page through every user account on the instance.
- **Query params:** `search` (default `""`, matches against email/name), `role` (optional filter — `admin` / `analyst` / `viewer`), `is_active` (optional bool filter), `page` (default `1`), `page_size` (default `25`), `sort_by` (default `"created_at"`), `sort_dir` (default `"desc"`).
- **Response:** `{"items": [{"id": str, "email": str, "full_name": str, "role": str, "is_active": bool, "created_at": str, "last_login_at": str | null}], "total": int, "page": int, "page_size": int}`
- **Example:**

```
curl "http://localhost:8000/api/v1/admin/users?role=analyst&is_active=true&page=1&page_size=25" \
  -H "Authorization: Bearer <REDACTED>"
```

`GET /api/v1/admin/users/stats`

- **Permission:** `user:manage` (admin only)
- **Purpose:** Aggregate user-base statistics for the admin console's overview panel.
- **Response:** `{"total_users": int, "active_users": int, "disabled_users": int, "by_role": {"admin": int, "analyst": int, "viewer": int}, "recent_logins": [{...}]}`

`GET /api/v1/admin/roles`

- **Permission:** `user:manage` (admin only)
- **Purpose:** Read-only view of the fixed role-to-permission matrix (the same `ROLE_PERMISSIONS` table reproduced in the "Authorization" section above), so the Roles & Permissions admin page never hardcodes a second copy of it that could drift out of sync with the real one.
- **Response:** `[{"role": "admin", "permissions": ["analysis:generate", "audit:read", ...]}, ...]` — one entry per role, permissions sorted alphabetically.
- **Example:** `curl http://localhost:8000/api/v1/admin/roles -H "Authorization: Bearer <REDACTED>"`

`POST /api/v1/admin/users`

- **Permission:** `user:manage` (admin only)
- **Purpose:** Create a new user account directly, with an explicit role — the normal way to create every account after the very first bootstrap admin (see §1's `/auth/register`).
- **Request body:** `{"email": "EmailStr", "password": "str (8-72 chars)", "full_name": "str (default \"\")", "role": "admin"|"analyst"|"viewer" (default \"analyst\")}`
- **Response (`201`):** the created user record.
- **Errors:** `409` if the email is already registered.
- **Example:**

```
curl -X POST http://localhost:8000/api/v1/admin/users \
  -H "Authorization: Bearer <REDACTED>" \
  -H "Content-Type: application/json" \
  -d '{"email": "new.analyst@example.com", "password": "<REDACTED>", "full_name": "New Analyst", "role": "analyst"}'
```

`PATCH /api/v1/admin/users/{user_id}`

- **Permission:** `user:manage` (admin only)
- **Purpose:** Update a user's display name and/or role.
- **Request body:** `{"full_name": "str | null", "role": "admin"|"analyst"|"viewer" | null}`
- **Errors:** `409` (`LastAdminError`) if this change would leave the instance with zero admins; `400` (`SelfRoleChangeError`) if an admin tries to change their *own* role (a mid-session self-lockout guard, not an independent security boundary); `404` if the `user_id` doesn't exist.
- **Example:**

```
curl -X PATCH http://localhost:8000/api/v1/admin/users/<user_id> \
  -H "Authorization: Bearer <REDACTED>" \
  -H "Content-Type: application/json" \
  -d '{"role": "analyst"}'
```

`POST /api/v1/admin/users/{user_id}/active`

- **Permission:** `user:manage` (admin only)
- **Purpose:** Enable or disable a user account. A disabled account is rejected on its very next request, not merely at its token's natural expiry.
- **Request body:** `{"is_active": bool}`
- **Errors:** `409` (`LastAdminError`) if disabling this user would leave zero active admins; `404` if the user doesn't exist.
- **Example:**

```
curl -X POST http://localhost:8000/api/v1/admin/users/<user_id>/active \
  -H "Authorization: Bearer <REDACTED>" \
  -H "Content-Type: application/json" \
  -d '{"is_active": false}'
```

`POST /api/v1/admin/users/{user_id}/reset-password`

- **Permission:** `user:manage` (admin only)
- **Purpose:** Administrator-initiated password reset. Increments the target user's `token_version`, which immediately invalidates every access and refresh token already issued to them — they are logged out everywhere on their very next request, not just unable to log in with the old password.
- **Request body:** `{"new_password": "str (8-72 chars)"}`
- **Errors:** `404` if the user doesn't exist.
- **Example:**

```
curl -X POST http://localhost:8000/api/v1/admin/users/<user_id>/reset-password \
  -H "Authorization: Bearer <REDACTED>" \
  -H "Content-Type: application/json" \
  -d '{"new_password": "<REDACTED>"}'
```

---

12. Security Assessment Toolkit — `/api/v1/security-assessment`

Not present in the prior internal API chapter, which explicitly scoped this router out. Added here in full after reading `backend/app/api/routes/security_assessment.py` and its service layer directly. This is the API behind active-scanning checks against a target — Nmap port/service scanning, DNS, TLS, HTTP-header inspection, vulnerability-intel lookups, and hash-based checks — gated by an explicit, mandatory scope/authorization confirmation on every single run, never implicit consent inherited from having created the underlying lookup. A dedicated router, deliberately not folded into `lookup.py`, mirroring `admin.py`'s precedent for a self-contained subsystem.

`GET /api/v1/security-assessment/profiles`

- **Full URL:** `http://localhost:8000/api/v1/security-assessment/profiles`
- **Permission:** `security_assessment:read` (admin, analyst, viewer)
- **Purpose:** List every tool/profile combination available to run — e.g. Nmap's `quick` / `standard` / `web` profiles — so a client can build a run request without hardcoding tool internals.
- **Response:** list of `{"tool_id": str, "tool_name": str, "profile_id": str, "name": str, "description": str, "supported_types": [str]}` .
- **Example:**

```
curl http://localhost:8000/api/v1/security-assessment/profiles \
  -H "Authorization: Bearer <REDACTED>"
```

```
[
  {"tool_id": "nmap", "tool_name": "Nmap Port/Service Scan", "profile_id": "quick", "name": "Quick scan", "description": "Top ports only, fast", "supported_types
]
```

`GET /api/v1/security-assessment/tool-health`

- **Permission:** `security_assessment:read` (admin, analyst, viewer)
- **Purpose:** Report whether each underlying scanning tool (e.g. the `nmap` binary) is actually available in this container right now — checked live at request time (e.g. via `shutil.which("nmap")`), not a static assumption.

- **Response:** list of `{"tool_id": str, "tool_name": str, "available": bool}`.

`POST /api/v1/security-assessment/{lookup_id}/run`

- **Full URL:** `http://localhost:8000/api/v1/security-assessment/{lookup_id}/run`
- **Permission:** `security_assessment:create` (admin, analyst)
- **Purpose:** Start an active-scan run against the target of an already-existing lookup. **This is asynchronous, not a blocking call:** the run is validated and persisted as `pending` / `running` and executed as a background task; this endpoint returns immediately with a run ID, and the caller must poll `GET .../runs` or `GET .../runs/{run_id}` (below) to observe progress and results.
- **Path param:** `lookup_id` (UUID) — the lookup whose seed IOC is the scan target.
- **Request body:**

Field	Type	Notes
`tool_ids`	`list[str]`	e.g. `["nmap", "dns", "tls"]`
`profile`	`str`	one profile ID, applied to every selected tool — e.g. `"quick"`
`target_confirmation`	`str`	the caller must retype the exact target value being assessed; checked server-side against the lookup's own `ioc_value`, never trusted at face value — the mandatory, explicit scope-confirmation step
`authorization_confirmed`	`bool`	default `false`; must be explicitly `true`

- **Response:** `{"run_id": str, "status": "pending"}`
- **Errors:** `404` `"No such lookup: <id>"`; `400` for every scope/validation failure — `"You must confirm you are authorized to test this target."` (authorization not confirmed), `"target_confirmation must exactly match this investigation's target."` (typo or mismatch), `"IOC type '<type>' has no network-addressable target to assess."` (e.g. a CVE or malware-family IOC, which has nothing to send traffic to), `"Active scanning is limited to 16 addresses (/28) or smaller -- this range has <n>."` (an oversized CIDR block), `"Unknown tool id: '<id>'"`, or `"Tool '<id>' does not support IOC type '<type>'."`.
- **Example:**

```
curl -X POST http://localhost:8000/api/v1/security-assessment/b3c1.../run \
  -H "Authorization: Bearer <REDACTED>" \
  -H "Content-Type: application/json" \
  -d '{
    "tool_ids": ["nmap", "tls"],
    "profile": "quick",
    "target_confirmation": "185.220.101.45",
    "authorization_confirmed": true
  }'
```

`GET /api/v1/security-assessment/{lookup_id}/runs`

- **Permission:** `security_assessment:read` (admin, analyst, viewer)
- **Purpose:** List every assessment run ever started against a given lookup's target, newest first, each with its full finding list.
- **Response:** list of `{"id": str, "lookup_id": str, "target": str, "tool_ids": [str], "profile": str, "status": "pending"|"running"|"completed"|"failed", "requested_by": str | null, "authorization_confirmed_at": str, "started_at": str | null, "completed_at": str | null, "error_message": str | null, "findings": [{"id": str, "tool_id": str, "finding_type": str, "severity": "info"|"low"|"medium"|"high"|"critical", "title": str, "description": str, "target_detail": str | null, "cve_ids": [str], "evidence": {}, "created_at": str}]}`.

`GET /api/v1/security-assessment/runs/{run_id}`

- **Permission:** `security_assessment:read` (admin, analyst, viewer)
- **Purpose:** Fetch one specific run by its own ID (rather than by lookup) — the natural endpoint to poll after starting a run via `POST .../run` above.
- **Response:** the same single run object as above.
- **Errors:** `404` `"Run not found"`.
- **Example:** `curl http://localhost:8000/api/v1/security-assessment/runs/<run_id> -H "Authorization: Bearer <REDACTED>"`

---

13. Executive Dashboard — `/api/v1/dashboard`

Both endpoints back the Executive Dashboard. Both are read-only aggregations over already-persisted data — neither triggers a new provider call, and neither persists anything.

`GET /api/v1/dashboard/kpis`

- **Full URL:** `http://localhost:8000/api/v1/dashboard/kpis`
- **Permission:** `dashboard:read` (admin, analyst, viewer)

- **Purpose:** Return the seven KPI tiles shown on the Executive Dashboard — every number a live database aggregate, never hardcoded.
- **Response:** `{"active_investigations": int, "critical_high_risk_iocs": int, "open_cases": int, "open_critical_cases": int, "avg_threat_score": number, "provider_health_percentage": number, "ai_success_rate": number | null}`. `critical_high_risk_iocs` and `avg_threat_score` are computed over a 30-day window; `provider_health_percentage` over a 24-hour window; `ai_success_rate` over a 30-day window and deliberately excludes `skipped_no_evidence` outcomes (a correct decision not to call the AI at all, not a failure) from both its numerator and denominator — it is `null`, not `0`, when there is no qualifying AI activity in the window at all.
- **Errors:** none explicit.
- **Example:**

```
curl http://localhost:8000/api/v1/dashboard/kpis \
  -H "Authorization: Bearer <REDACTED>"
```

```
{
  "active_investigations": 3,
  "critical_high_risk_iocs": 41,
  "open_cases": 12,
  "open_critical_cases": 2,
  "avg_threat_score": 34.7,
  "provider_health_percentage": 96.4,
  "ai_success_rate": 98.1
}
```

`GET /api/v1/dashboard/executive-summary`

- **Full URL:** `http://localhost:8000/api/v1/dashboard/executive-summary`
- **Permission:** `dashboard:read` (admin, analyst, viewer) — reuses the same permission as `/kpis`; no new permission was introduced for this endpoint.
- **Purpose:** Return a short AI-generated executive narrative, grounded exclusively in the same seven KPI values `GET /dashboard/kpis` returns — the AI is only ever handed those numbers as given facts and never computes or restates one independently.
- **Response:** `{"summary": str, "source": "ai" | "template_fallback", "kpis": {...same object as GET /dashboard/kpis...}}`. `source` honestly discloses which path produced the text: `"template_fallback"` on any AI failure that survives one validation retry (backend unreachable, or a response that fails schema validation twice) — the fallback sentence is still built directly from the real KPI numbers, never an error message and never fabricated.
- **Errors:** none explicit — an AI failure is reported via `"source": "template_fallback"` inside a normal `200` response, not an HTTP error status.
- **Example:**

```
curl http://localhost:8000/api/v1/dashboard/executive-summary \
  -H "Authorization: Bearer <REDACTED>"
```

```
{
  "summary": "Provider health is strong at 96% and the AI pipeline succeeded on 98% of eligible investigations over the last 30 days. Two critical cases remain o
  "source": "ai",
  "kpis": {"active_investigations": 3, "critical_high_risk_iocs": 41, "open_cases": 12, "open_critical_cases": 2, "avg_threat_score": 34.7, "provider_health_perc
}
```

Unversioned utility endpoints

Three endpoints are defined directly in `backend/app/main.py`, outside `app/api/routes/`, with no `/api/v1` prefix and no permission check of any kind:

`GET /health`

- **Full URL:** `http://localhost:8000/health`
- **Purpose:** Liveness probe — this is what the Windows installer's "Service Status" shortcut and any container-orchestration health check actually poll.
- **Response:** `{"status": "ok", "service": "HORIZON GRID"}`

`GET /network-info`

- **Full URL:** `http://localhost:8000/network-info`
- **Purpose:** Return the LAN IP address the Windows Setup Wizard detected at install time, plus the configured frontend/backend host ports — used by the frontend to render "reach this instance at" guidance for other machines on the same network. This is a point-in-time snapshot from the wizard's last run, not a live re-detection: a container can never reliably discover the Windows host's real LAN-facing IP from inside itself (confirmed directly — self-detection from inside the container returns Docker's own bridge address, and `host.docker.internal` resolves to Docker Desktop's internal gateway, neither of which is the host's real interface).
- **Response:** `{"detected_lan_ip": str | null, "frontend_port": int, "backend_port": int}`
- **Example:** `curl http://localhost:8000/network-info`

`GET /metrics`

- **Full URL:** `http://localhost:8000/metrics`
- **Purpose:** Prometheus-format metrics, exposed by `prometheus-fastapi-instrumentator` — request counts/latencies by route, standard process metrics. Intended for a metrics scraper, not a human reading JSON.
- **Response:** `text/plain`, Prometheus exposition format.

Both `/health` and `/network-info` share the same trust model: unauthenticated, and deliberately so, since neither exposes anything beyond what's needed for basic liveness/reachability checks before a caller has ever obtained a token.

Appendix: endpoint count by router

Router	Endpoints	Permission strings used
`auth.py`	4	none (public) / implicit valid-token check on `/me`
`lookup.py`	6	`lookup:create`, `lookup:read`, `lookup:export`
`providers.py`	2	`dashboard:read`, `provider:manage`
`ai_config.py`	2	`provider:manage`
`analysis.py`	10	`evidence:read`, `analysis:generate`, `copilot:query`
`hunting.py`	2	`hunting:generate`
`pivot.py`	1	`lookup:read`
`basket.py`	5	`basket:manage`
`cases.py`	8	`case:read`, `case:create`, `case:write`, `case:close`
`runtime.py`	10	`provider:manage`, `lookup:read`, `audit:read`
`admin.py`	7	`user:manage`
`security_assessment.py`	5	`security_assessment:read`, `security_assessment:create`
`dashboard.py`	2	`dashboard:read`
Total	**64**	**18 distinct permission strings**

Related reference material, covered in full elsewhere in this documentation set rather than repeated here: the exact deterministic-scoring formula behind `risk.overall_risk_score` / `confidence_score` / `malicious_probability` (Threat Scoring reference), the complete RBAC/credential-security threat model (Security Architecture chapter), and the Security Assessment Toolkit's tool-by-tool internals (Security Assessment Toolkit chapter).